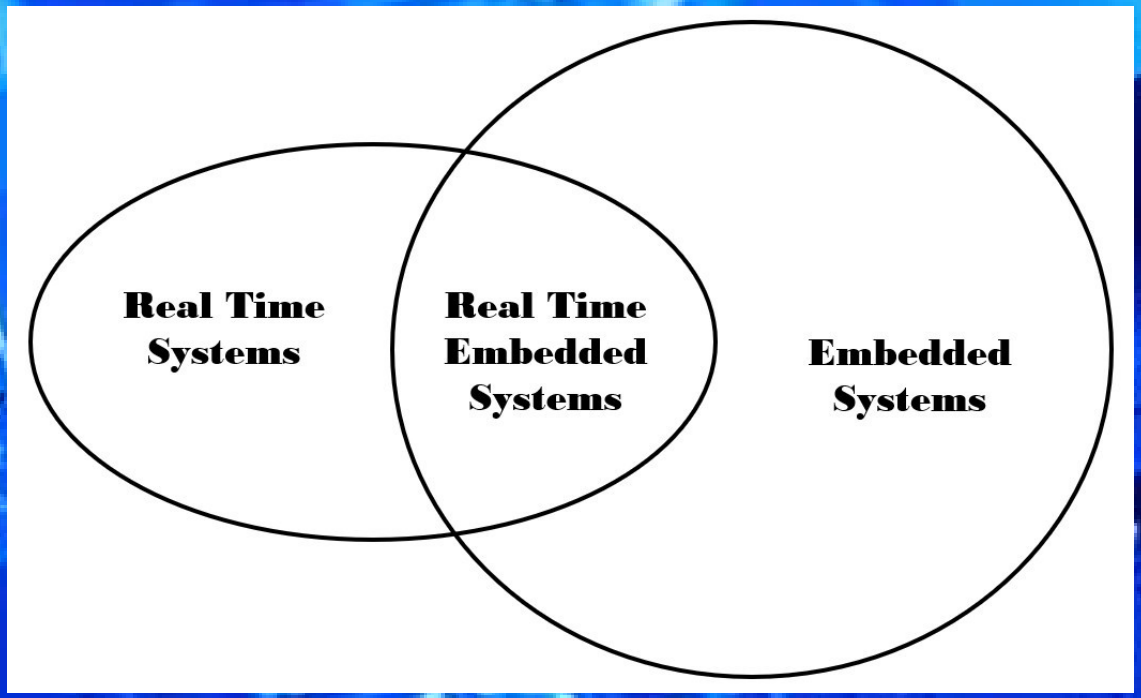
EEE3096S

Embedded Systems II



Terms & Concepts

Real Time Systems
Hard and Soft Deadlines

Lecture L2

Embedded Systems II

Lecturer Best N.

Electrical Engineering
University of Cape Town

Outline

- Embedded Systems and Real-Time Systems
- Characteristics of Real-Time Systems (RTS)
- Taxonomy of Real-Time Systems
- *ACTIVITY – chat, online quiz & answers*
- Aspects of Hard vs Soft RTS
- IoT, IIoT and some other trendy terms
- Optimization of Embedded Systems.

Embedded Systems (ES)

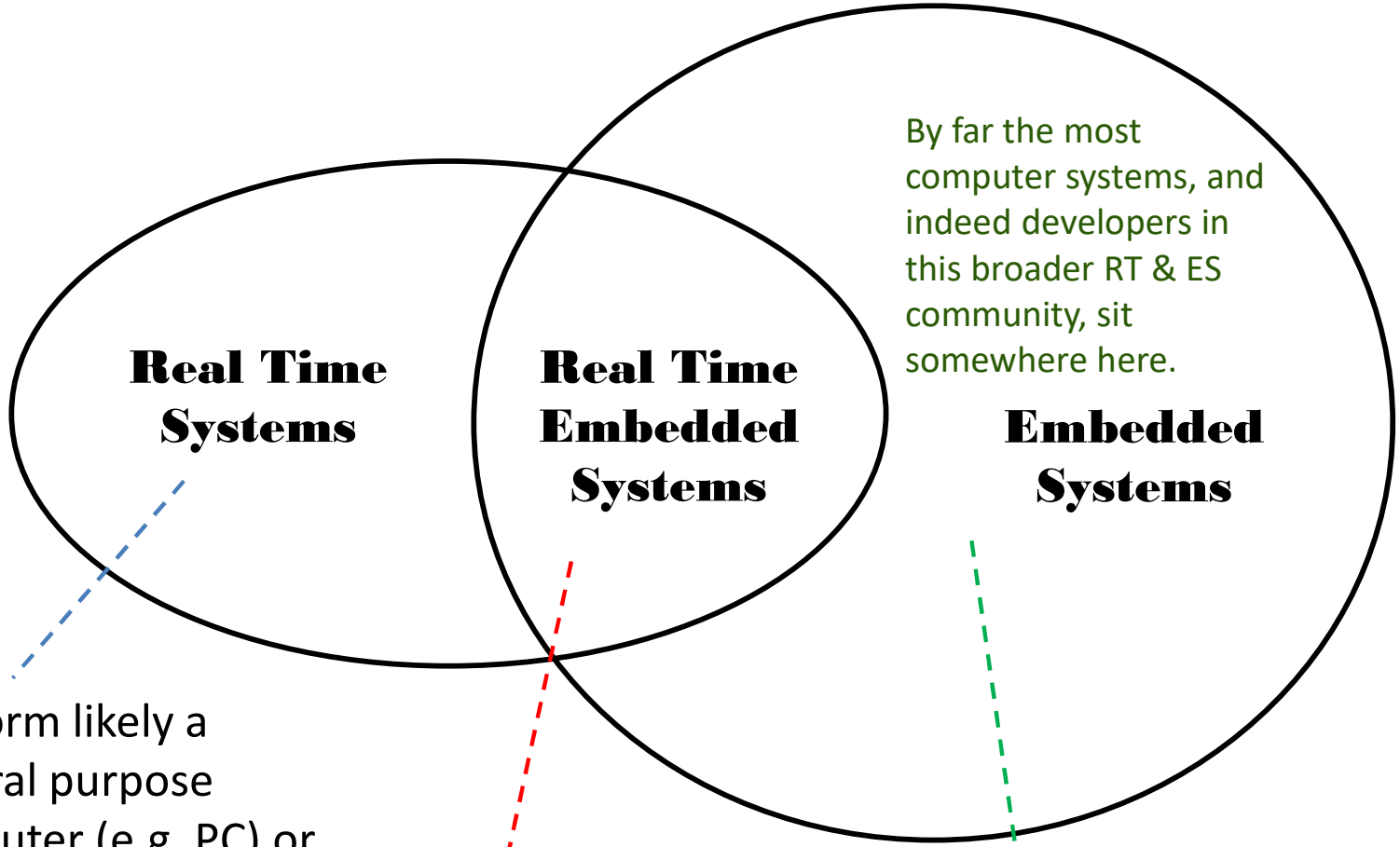
vs.

Real-Time Systems (RTS)

vs.

Real-Time Embedded Systems (RTES)

ES vs RTS vs RTES



Platform likely a general purpose computer (e.g. PC) or resides in the cloud, not much choice on hardware design.

Can be a challenge to tame these beast (by 'tame' I mean ensure hard deadlines are ensured).

A special-purpose computer built into a larger system (you know what these are already!)

Examples of ES vs RTS vs RTES

- Real Time Embedded
  - Flight control system
  - Nuclear reactor control
  - Basically any safety critical system
  - GPS
  - MP3 player
  - Mobile phone
  - Particle detection trigger and sampling for physics experiments

Airbus A320

- Real Time, but not Embedded:
  - Stock trading system
  - Skype
  - Pandora
- Embedded, but not Real Time:
  - Washing machine, refrigerator, etc.

Reference source: "Introduction to Real Time Systems" A Ledeczi, EECE354. Image source: Wikimedia Commons

Characteristics of Real-Time Systems

The main factors of an RT system are the following…

- Event-driven
  - The system is reactive to inputs or events (i.e. external stimuli)
- High cost of failure
  - e.g. damage to costly equipment or environment or even loss of life.
- Concurrency/multiprogramming
  - Doing multiple things at once, or actions occur close together in time
- Stand-alone/continuous operation
  - Often needs to run without depending on operation of other devices, needs to run for long periods without fail.
- Reliability/fault-tolerance requirements
  - Needs to keep running consistently for long periods, and often needs to keep running even in the case of faults or failing components (at least to some extent – e.g. graceful degradation or 'fail soft')
- Predictable behavior (this is usually the most important aspect)
  - Clear understanding of how and *when* system reacts in any given situation

RT Terminology

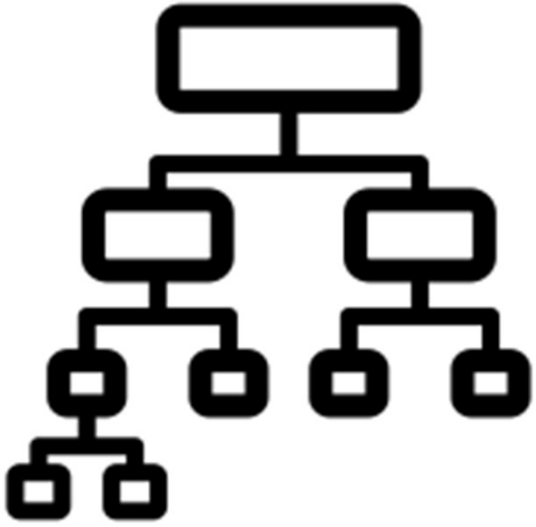
- **Hard real-time:** systems where it is absolutely imperative that responses occur within the required deadline. (e.g. flight control systems).
- **Soft real-time:** systems for which deadlines are important but which will still function correctly if deadlines are occasionally missed (e.g. a weather data logging system).

RT Terminology

- **Real real-time:** systems that have at least one hard real-time and for which the response times are usually very short. e.g. Missile guidance system.
- **Firm real-time:** systems that are soft real-time but for which there is no benefit from late delivery of service. For example GPS

In reality, a system is more likely a combination of above cases; more accurately viewed as a cost function being associated with missing each deadline.

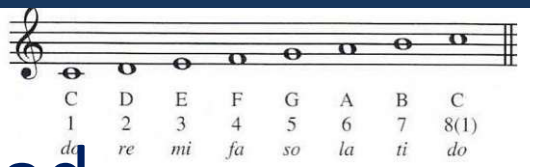
Taxonomy of Real-Time Systems



Static vs. Dynamic RT Systems

- Static:
  - **Task arrival times are finely predicted**
  - Allows static analysis (i.e. can be fully analysed offline at compile-time or before runtime)
  - Can design around well-balanced resource usage (i.e., minimize idle time, optimize system processing costs)
  - Example, Industrial control systems
- Dynamic:
  - **Arrival times of tasks may be unpredictable**
  - Static (compile-time) analysis often not possible or very difficult (or too costly) to model accurately.
  - Needing to avoid over-simplifying assumptions (e.g., assuming all tasks are independent, could overlap, which could in practice be very unlikely).
  - Often makes for more difficult designs.
  - Example, missile guidance system

Periodic or Synchronous Design



- **Periodic** = each task (or task group) executes repeatedly at specific period.
- Well-suited to hard RT and simplifies static analysis.

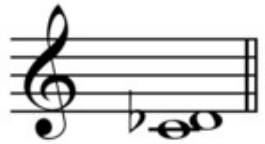
Periodic or even harmonically periodic design is music to my ears… as they're usually easier to design ☺

- Matches character of many real problems.
- Could have tasks with deadlines that are: smaller, equal, or greater than their period.
- Single- or Multi-periodic rate
  - Single rate: One period in the system, simple but inflexible (e.g. often used in wireless sensor nets)
  - Multi rate: Multiple periods; but usually simplified as harmonics to simplify the design

Note: don't assume this implies system inputs will be polled, may be a dangerous assumption as the system may need near simultaneous inputs or rapid input reads but ensuring hard deadline met likely at a higher level (some inputs may be asynchronous to satisfying deadlines).

Aperiodic Design

- An **aperiodic, or 'sporadic', system is** designed around being **asynchronous** or reactive.
- Creates a dynamic situation (periods between events/responses may become very varied)



Some might even call these systems dissonant (at least if they're a bit flaky).

Note: don't assume this implies the system inputs will be polled, this may be a dangerous assumption as the system may need to support near simultaneous inputs or rapid input reads but the higher level hard deadline may be at a higher level, some inputs possibly asynchronous to operations to satisfy deadlines.

QUIZ!

Activity A: Work together in a team to decide on some responses. Can have ~5 minutes, don't need to answer all.

Answers on next slide…

ANSWERS

Q1: If the system has 2 hard deadlines and 98 soft deadline, is it a soft RTS? A: ~~Yes~~ / **No** ?

Q2: Which feature below sounds to be more in an "Aperiodic Hard Real-time System"

>**a.** Interrupts that is coded to ensure event is responded to within 5ms

b. For-loop ensures every input checked every 5ms

c. Goto function that jumps to address 0 to crash the system at some stage.

Q3: I have a system comprises three computers, one hard RTS, one soft RTS, and the third is a bit of both. What is the system as a whole classified as?

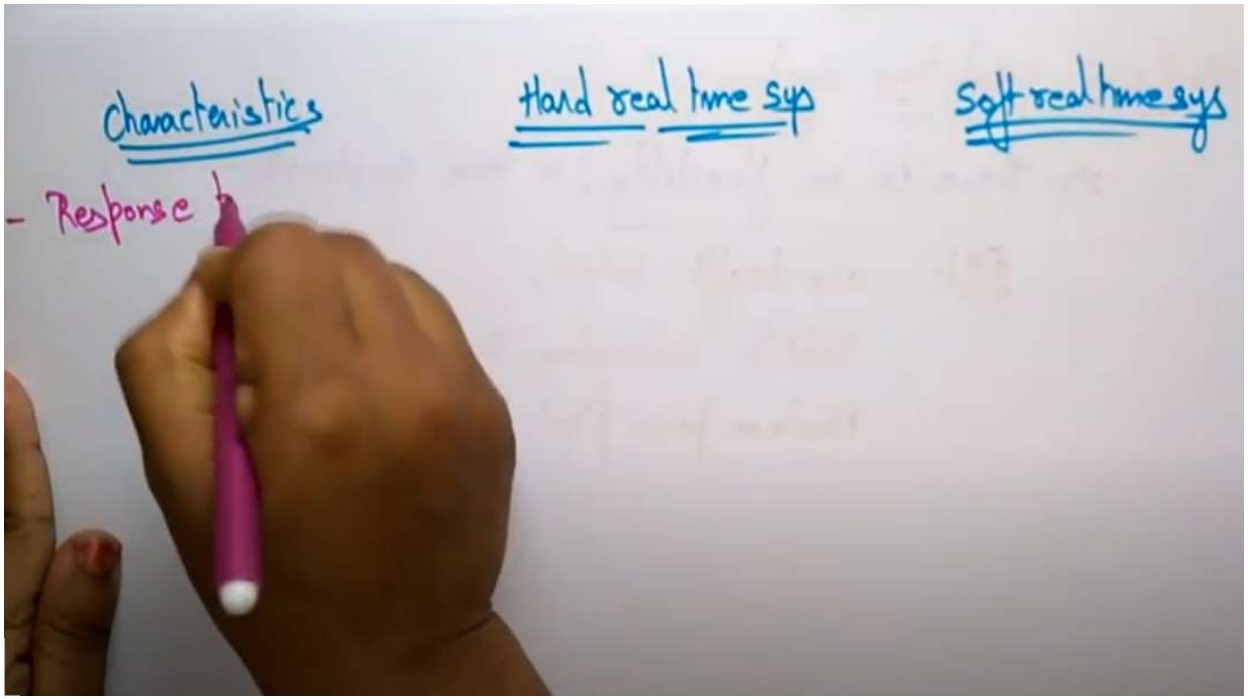
>**a.** A hard RTS

b. A soft RTS

c. Neither hard nor soft RTS, as 3rd system is either

Answers on next slide...

Hard vs Soft Deadlines: Training Video



Hard and Soft Realtime System Characteristics

https://youtu.be/1DAanYxJVTQ

See Vula/Lectures/Week1/VideoClips/Hard and Soft Realtime system Characteristics.mp4

Watch this video, a summary of main points on next slide.

Hard vs Soft Realtime System Characteristics

Characteristic	Hard RTS	Soft RTS
Response time	Hard required	Soft required
Peak load performance	Predictable	Degradable
Control of pace	Environment	Computer
Safety	Often critical*	Non-critical
Size of data files	Usually small/medium	Can be large
Redundancy type	Active (concurrent)	Check-point recovery (e.g. can rollback and fix things)
Data integrity	Short-term	Long-term
Error detection	Autonomous and usually rapid in situ (can't reply on human intervention)	Potentially user-assisted

* The concept of 'safety-critical systems' are particularly pertinent here:

Defn **Safety-Critical System (SCS)**: or 'life-critical system' is a system whose failure could result in one (or more) of the following outcomes: death or serious injury to people loss or severe damage to equipment/property or precious environment.

Internet of Things (IoT)
&
Industrial IoT (IIoT)

Super Brief Intro to these… well get into more IoT issues related to ES later in the course.

Further depth of IoT networking, in relation to embedded systems, follows in lecture 3.

IoT

- Main insights are:
  - IoT = the billions of physical embedded devices around the world that are connected to the internet, all collecting and delivering data.
  - This is possible from super cheap computer chips and ubiquity of wireless networks… together making it possible to turn (almost) anything into a part of the IoT.
  - The massive connectivity of these sensors and other IoT devices brings digital intelligence to devices that would be otherwise be (independently) dumb, enabling them to communicate real-time data without needing implicit involvement of humans.
  - IoT is making the fabric of the world around us smarter and more responsive, merging the digital and physical universes.

Industrial IoT (IIoT)

- IIoT can be considered the extension or specialization of IoT devices towards industry application.
- The main focus of IIoT is:
  - Machine-to-Machine (M2M) communication
  - Big data
  - Machine learning (to optimize industrial processes)
- IIoT is predominantly about enabling industries (or more generally businesses) to have better monitoring, efficiency, control and reliability in their operations.
- The IIoT encompasses many industrial applications, including: robotics, software-defined production processes, logistics, medical devices, among others.

Optimization of Embedded Systems

(brief prelude, more on this in the last week)

ES Design Optimization Metrics

- Optimization of an ES, especially a RTES, can be complex and involves a potential multitude of conflicting dimensions (i.e. features may work against each other) e.g.:
  - more memory → higher cost
  - less power → slower operation
- Usually, and what ES engineers often spend much time discussing, is the 'optimization metrics' they are using or their optimization schedule (if not 'optimization matrix' that is used to work out satisfactory compromises).

ES Optimization Design Metrics

Defn "Design Metric": a measurable feature of an embedded system's design, e.g. its performance, cost, time for implementation, safety, etc.

As mentioned, these are typically conflicting requirements i.e. optimizing one doesn't necessary optimize the other – indeed may instead reduce it.

As you have probably now realized, with your existing understanding of engineering, there could be masses of such optimization metrics….

But the 'usual suspects' are the following (not in priority order) :

1. NRE cost (nonrecurring engineering cost)
2. Unit cost
3. Size
4. Performance
5. Power Consumption
6. Flexibility
7. Time-to-prototype
8. Time-to-market
9. Maintainability
10. Correctness

(although you're probably aware of what each of these aspects are, the following slides gives a brief summarize for each point).

Usual ES Optimization Design Metrics

- NRE cost (nonrecurring engineering cost) =

  The one-time (mostly labour) cost of designing the system.
- Unit cost

  The monetary cost of manufacturing each copy of the system, excluding the NRE cost.
- System Size

  The physical space required by the system, often measured in bytes for software, and gates or transistors for hardware .
- Performance

  The execution time of the system.
- Power Consumption

  Amount of power system consumes, may determined the battery it needs and/or cooling requirements.

Usual ES Optimization Design Metrics

- Flexibility

  The ability to change the functionality of the system without incurring heavy NRE cost (software is typically considered very flexible).
- Time-to-prototype

  Time needed to build a working version of the system. May be bigger or more expensive than the final implementation, but it used to verify the system's usefulness and correctness, and refine its design.
- Time-to-market

  Time to develop system to point it can be released and sold. Main contributors = design time + manufacture time + thorough testing time. This metric is especially demanding in recent years*.
- Maintainability

  The ability to modify the system after its initial release, especially by designers who did not originally design the system.
- Correctness

  Important measure of the confidence: is the functionality correct? Sometimes can't compromise on this, especially for a safety-critical system.

* Early adoption by target market could mean big improvement to system's profitability.

Optimizing Embedded Systems: Final Point

- Although we're going to see more about optimizing embedded systems… you'd need to complete most other aspects of this course before it makes sense to dive into ES optimization.
- The act of ES optimization is considered by many as more an "Art" than a science or design methodology (as captured by Jack Ganssle's famous book "The Art of Designing Embedded Systems")

More on embedded system optimization in last week of course.

End of Lecture

The Next Episode…

Lecture L03

Wednesdays Lecture

- Development & Execution Environment
- Cross-Compiling Toolchain
- Make, StdC, Debugging
- State Machines